BUNUS.APP

Product Requirements, Functional Specification
& Technical Implementation Brief

Dokumen untuk coding agent

Versi 1.0 · 3 November 2026 · Bahasa Indonesia

Instruksi teknis paling penting: **PHP dilarang.** Implementasi menggunakan Node.js LTS, TypeScript, NestJS, Vue 3, MySQL 8, Prisma, dan PWA.

Tagline: **Every visit brings a little more.**

Brand direction: Modern · Friendly · Premium · Trustworthy

Product Requirements, Functional Specification & Technical Implementation Brief

Dokumen untuk coding agent — versi 1.0 — 3 November 2026

0. Instruksi utama untuk coding agent

Bangun **Bunus.app**, sebuah multi-tenant SaaS Digital Loyalty Platform untuk merchant di Indonesia dan pasar Asia. Platform mendukung program loyalty berbasis **Stamp** dan fondasi untuk **Points**. Customer memiliki satu Global Customer Account yang dapat bergabung dengan banyak merchant, sedangkan data, saldo, ledger, reward, dan histori setiap merchant harus terisolasi.

Ketentuan teknologi yang wajib

- **JANGAN GUNAKAN PHP.** Tidak boleh menggunakan Laravel, CodeIgniter, WordPress/PHP backend, atau PHP sebagai runtime aplikasi.
- Backend: **Node.js LTS + TypeScript + NestJS**.
- Frontend: **Vue 3 + TypeScript + Vite**.
- Database: **MySQL 8**.
- ORM: **Prisma** (migrations wajib disimpan di Git).
- UI: responsive web/PWA.
- Animasi stamp: **Lottie/DotLottie + CSS transitions**.
- Ikon: **Lucide**.
- Font: **Poppins** untuk heading/brand; **Inter** untuk body, angka, tabel, dan data.
- Arsitektur: **modular monolith**, bukan microservices untuk MVP.
- Deployment: harus portable ke Linux VPS dan Docker. cPanel Node.js-compatible boleh dipertimbangkan sebagai opsi hosting, tetapi jangan membuat arsitektur bergantung pada fitur khusus cPanel.
- Semua secret/config melalui environment variables. Jangan commit `.env`.
- Gunakan UTC untuk timestamp penyimpanan; tampilan mengikuti timezone merchant/customer sesuai kebutuhan.

Stack ini adalah baseline implementasi yang dipilih untuk brief ini. Jangan menggantinya dengan PHP atau menambahkan Python runtime/Emergent dependency.

Cara kerja coding agent

1. Baca dokumen ini sebelum membuat kode.
2. Buat rencana implementasi dan daftar asumsi yang belum ditetapkan.
3. Buat project skeleton, linting, formatting, test runner, migration workflow, dan `.env.example`.
4. Implementasikan domain/business rules sebelum mempercantik UI.
5. Jangan menganggap mock data sebagai fitur selesai.
6. Semua operasi yang mengubah stamp, poin, reward, redemption, subscription, atau referral wajib memiliki validasi server-side, transaksi database, idempotency, dan audit trail yang sesuai.
7. Jangan pernah mempercayai data tenant, role, harga, saldo, status pembayaran, atau eligibility yang dikirim langsung dari browser.
8. Jangan membuat fitur yang terlihat sukses sebelum backend mengonfirmasi commit.
9. Tulis unit, integration, dan E2E tests untuk acceptance criteria pada dokumen ini.

10. Jika ada keputusan bisnis yang belum ditetapkan, implementasikan dengan konfigurasi yang eksplisit dan aman; jangan mengarang kebijakan secara diam-diam.

1. Product vision dan positioning

Nama produk: Bunus

Domain utama: `bunus.app` (dipilih oleh product owner; ketersediaan/registrasi domain perlu dikonfirmasi secara terpisah).

Kategori: Multi-merchant Digital Loyalty SaaS.

Target pasar: Indonesia terlebih dahulu, dengan desain yang dapat diperluas ke Asia/internasional.

Brand tagline: “Every visit brings a little more.”

Product description: Platform SaaS yang memungkinkan bisnis membuat dan mengelola program loyalty digital—stamp, points, rewards, referral, membership, dan aktivitas customer—tanpa harus membangun aplikasi sendiri.

Tujuan produk

- Merchant dapat membuat loyalty program dan mengelola customer dari dashboard.
- Customer memiliki satu akun global untuk bergabung ke berbagai merchant.
- Customer dapat melihat benefit, stamp, points, reward, dan histori per merchant.
- Staff dapat memvalidasi customer dan memberikan stamp/reward secara aman.
- Pemilik Bunus dapat mengelola merchant, paket subscription, pembayaran, dan status layanan melalui Super Admin Dashboard.
- Sistem dapat berkembang dari MVP menjadi SaaS multi-tenant tanpa rewrite besar.

Prinsip produk

- One platform, multi-merchant.
- One global customer identity, separate merchant memberships.
- Merchant autonomy: setiap merchant dapat mengatur aturan program sendiri.
- Ledger-first: saldo adalah hasil perhitungan ledger, bukan angka yang diedit langsung.
- Secure by default: tenant isolation, RBAC, audit logs, idempotency.
- Portable infrastructure: mudah dipindahkan dari managed hosting/cPanel Node ke VPS.
- Mobile-first untuk customer PWA; desktop-first/responsive untuk dashboard.

2. Aktor, role, dan hak akses

2.1 Roles

1. **Platform Super Admin**

- Pemilik/operator Bunus.
- Mengelola merchant, plan, subscription, invoice, payment, status layanan, konfigurasi platform, audit, dan support.

2. **Merchant Owner**

- Pemilik merchant.

- Mengelola profil merchant, loyalty programs, cabang, staff, rewards, referral, analytics, subscription, dan pengaturan.

3. **Merchant Manager**

- Mengelola operasional merchant sesuai permission yang diberikan.

4. **Branch Manager**

- Mengelola staff dan aktivitas pada cabang yang ditugaskan.

5. **Staff**

- Melakukan validasi membership, stamp issuance, dan reward redemption sesuai permission.

6. **Customer**

- Satu global account untuk semua merchant.

- Memiliki membership terpisah di setiap merchant/program.

2.2 Access control

- Gunakan RBAC dengan permission granular; role bukan satu-satunya kontrol.
- Authorization harus divalidasi di backend untuk setiap endpoint.
- Staff dibatasi pada branch yang ditugaskan.
- Merchant user hanya dapat mengakses tenant miliknya.
- Customer hanya dapat melihat membership, stamp, points, reward, dan histori miliknya sendiri.
- Super Admin memiliki akses lintas tenant melalui jalur administrasi yang diaudit.
- Jika fitur “View as Merchant/Impersonation” dibuat, harus memiliki alasan, durasi/session terbatas, identitas admin tercatat, dan audit log. Jangan pernah meminta atau menampilkan password merchant.

3. Multi-tenant dan Global Customer Account

3.1 Model tenant

Setiap merchant adalah tenant terpisah. Data merchant wajib mempunyai `merchant_id` atau relasi tenant yang setara. Query tenant harus selalu dibatasi oleh tenant context dari sesi/token yang sudah diverifikasi server.

Jangan menerima `merchant_id` dari request sebagai bukti otorisasi. Server harus memastikan user memang memiliki akses ke merchant tersebut.

3.2 Global Customer Account

Customer mendaftar satu kali dan dapat menjadi member di banyak merchant.

Contoh:

- Customer memiliki akun global dengan email terverifikasi.
- Bergabung dengan ABC Coffee: membership dan stamp ABC Coffee.
- Bergabung dengan XYZ Salon: membership dan points XYZ Salon.
- Kedua merchant tidak dapat melihat data loyalty satu sama lain.

Struktur konseptual:

- `customers`: identitas global customer.

- `customer_auth_identities`: email/phone/credential/verification.
- `merchant_memberships`: relasi customer dengan merchant.
- `program_memberships`: relasi customer dengan program tertentu bila satu merchant punya lebih dari satu program.

Membership harus memiliki status sendiri, tanggal bergabung, sumber pendaftaran/referral, dan preferensi komunikasi yang relevan.

3.3 Customer PWA routes

Domain utama menggunakan satu domain dan path untuk MVP:

- `https://bunus.app/` — marketing/public website.
- `/member` — customer home dan membership list.
- `/member/login`, `/member/register`, `/member/verify`.
- `/member/memberships/:membershipId` — detail membership.
- `/member/rewards` — reward yang tersedia/terbuka.
- `/member/history` — histori customer.
- `/merchant` — merchant dashboard.
- `/staff` — staff dashboard.
- `/admin` — Super Admin dashboard.
- `/m/:merchantSlug` — public merchant loyalty landing/join page.
- `/api/v1` — API.

Subdomain atau custom domain merchant dapat menjadi fase lanjutan. Jangan membuat domain terpisah per merchant pada MVP.

4. Customer experience dan authentication

4.1 Registrasi dan verifikasi

Rekomendasi awal:

- Registrasi menggunakan email dan password.
- Verifikasi email melalui verification link atau OTP.
- Nomor telepon opsional pada awal registrasi; dapat diverifikasi kemudian.
- WhatsApp OTP menjadi opsi/fallback/fitur tambahan.
- SMS OTP menjadi fallback tambahan jika diperlukan.
- Jangan memaksa customer menggunakan WhatsApp atau SMS untuk membuat akun.

Email sender dapat menggunakan domain sendiri, misalnya `noreply@bunus.app`. Email OTP dapat dibuat oleh backend sendiri; third-party provider tidak wajib untuk menghasilkan OTP. SMTP/API provider digunakan untuk delivery email. Buat provider abstraction agar SMTP hosting dapat diganti Resend, Amazon SES, Brevo, atau provider lain tanpa mengubah auth flow.

4.2 OTP/email security

- OTP 6 digit yang dibuat dengan cryptographically secure random generator.

- OTP disimpan dalam bentuk hash, bukan plaintext.
- Masa berlaku default 5 menit (konfigurabel).
- One-time use; setelah berhasil digunakan, OTP tidak dapat digunakan lagi.
- Rate limit per account, IP, dan email/nomor.
- Cooldown resend awal 60 detik (konfigurabel).
- Batasi percobaan salah; contoh default 5 kali, lalu cooldown/lock sementara.
- Jangan bocorkan apakah email terdaftar melalui error yang terlalu spesifik pada flow sensitif.
- Verification link bersifat one-time dan memiliki expiration.
- Password di-hash dengan Argon2id atau bcrypt dengan konfigurasi aman.
- Session menggunakan secure, HttpOnly, SameSite cookies atau strategi token yang aman; hindari penyimpanan token sensitif di localStorage.
- Implementasikan password reset dan logout/session revocation.

4.3 Global customer home

Customer home menampilkan semua merchant membership dalam satu akun:

- Merchant logo/nama.
- Program type (Stamp/Points).
- Stamp progress atau points balance.
- Reward yang sudah terbuka.
- Reward yang akan segera expired.
- Recent activity.
- Join new program via QR/link.
- Notification center.

Setiap membership card hanya menampilkan data merchant yang bersangkutan.

4.4 Join merchant

Customer dapat bergabung melalui:

1. QR code merchant.
2. Public link `/m/:merchantSlug`.
3. Referral link.

Jika customer belum login, simpan join intent secara aman, arahkan ke login/register, lalu lanjutkan join setelah autentikasi. Pastikan referral attribution tidak hilang selama proses tersebut.

5. Loyalty Engine: Stamp dan Points

Bangun fondasi Loyalty Engine yang mendukung dua tipe program, tetapi MVP dapat meluncurkan Stamp Program lebih dahulu. Jangan membuat stamp dan points menjadi satu saldo campuran.

5.1 Loyalty Program

Setiap merchant dapat memiliki satu atau beberapa loyalty program. Program mempunyai:

- `program_type`: `STAMP` atau `POINTS`.
- Nama, deskripsi, status, tanggal aktif.
- Earning rules.
- Expiration rules.
- Reward catalog.
- Terms and conditions.
- Branch applicability.
- Program timezone/currency jika relevan.

5.2 Stamp program

Konsep awal:

- Default 1 qualifying visit/transaction = 1 stamp.
- Merchant dapat mengatur earning method:

- 1 qualifying visit = 1 stamp.

- Berdasarkan nominal transaksi (misalnya setiap Rp50.000 = 1 stamp).

- Konfigurasi lain hanya jika disetujui product owner.

- Jumlah stamp untuk membuka reward ditentukan merchant.
- Stamp dapat diberikan oleh staff melalui Staff Dashboard.
- Customer dapat melihat progress secara real-time/near-real-time.
- Stamp issuance harus idempotent dan atomic.

5.3 Stamp expiration — wajib configurable per merchant/program

Pilihan expiration:

1. **Never expire** — stamp tidak kedaluwarsa.
2. **Duration** — setiap stamp berlaku selama N hari/minggu/bulan sejak diterbitkan.
3. **Fixed date** — stamp yang termasuk aturan tersebut kedaluwarsa pada tanggal tertentu.

Rekomendasi default policy:

- Aturan baru berlaku untuk stamp baru.
- Stamp lama tetap mengikuti policy snapshot saat diterbitkan.
- Jika merchant ingin mengubah aturan lama, lakukan migration/adjustment eksplisit, dengan audit trail dan pemberitahuan customer sesuai kebijakan.
- Simpan `issued_at`, `expires_at`, policy/version reference, dan status pada ledger entry/lot.
- Jangan menyimpan hanya satu `expires_at` di membership untuk semua stamp jika duration dihitung per penerbitan.

Contoh:

- Stamp A diterbitkan 1 November, expired 30 Januari.
- Stamp B diterbitkan 15 November, expired 13 Februari.
- Stamp A yang expired tidak boleh menghapus stamp B.

Merchant dapat mengatur expiration reminder, misalnya 7 hari atau 3 hari sebelum expired. Reminder harus mengikuti consent dan channel preference customer.

Expired stamp:

- Tidak lagi dihitung dalam reward progress.
- Tidak boleh diredeem.
- Tidak boleh dihapus dari ledger.
- Buat expiration ledger event/entry agar histori dapat diaudit.
- Saldo/progress yang tampil adalah hasil perhitungan stamp aktif yang belum expired.

5.4 Points program

Fondasi points disiapkan, aktivasi dapat menyusul setelah Stamp MVP stabil.

Contoh:

- Rp10.000 transaksi = 1 point.
- Customer mengumpulkan 1.000 points.
- 1.000 points dapat ditukar voucher tertentu.

Points memiliki earning rule, balance, expiration policy, redemption, adjustment, dan ledger sendiri. Expiration dapat menggunakan lot/batch per earning event agar poin yang lebih baru tidak otomatis expired bersama poin lama, kecuali merchant memilih fixed-date policy.

5.5 Ledger-first architecture

Ledger adalah source of truth. Jangan hanya melakukan `balance = balance + 1` tanpa ledger.

Jenis ledger events minimum:

- `EARN` / `STAMP_ISSUED`
- `REDEEM` / `POINTS_REDEEMED`
- `EXPIRE`
- `ADJUSTMENT`
- `REVERSAL`
- `REFERRAL_REWARD`

Persyaratan:

- Ledger immutable secara prinsip; koreksi menggunakan reversal/adjustment entry, bukan mengedit histori.
- Setiap entry punya actor, source, reference, timestamp, tenant/program/member context.
- Gunakan DB transaction.
- Gunakan unique constraints/idempotency keys.
- Gunakan row locking atau mekanisme concurrency control untuk mencegah double issue/redeem.
- Saldo boleh berupa cached projection, tetapi harus dapat direkonsiliasi dari ledger.
- Jangan pernah membiarkan saldo negatif.

6. Stamp QR, issuance, dan redemption

6.1 QR flow

Dua konteks QR:

1. Customer membership QR: customer menunjukkan QR di smartphone untuk di-scan staff.
2. Merchant join QR: customer scan QR merchant untuk membuka halaman join.

Untuk issuance, staff harus login dan punya permission. QR customer sebaiknya menggunakan short-lived/dynamic token, bukan customer ID statis yang dapat disalahgunakan.

QR token:

- Ditandatangani/opaque token dengan expiration pendek.
- Tidak berisi PII sensitif.
- One-time atau dibatasi penggunaan sesuai flow.
- Backend memvalidasi token, membership, merchant, branch, staff permission, dan status subscription.

6.2 Stamp issuance

Alur:

1. Staff login.
2. Pilih/terdeteksi branch.
3. Scan customer membership QR.
4. Backend memvalidasi token dan membership.
5. Staff memasukkan nominal transaksi jika earning rule berbasis amount.
6. Backend menghitung stamp berdasarkan earning rule server-side.
7. Backend menulis ledger entry dalam DB transaction.
8. Backend mengembalikan hasil authoritative.
9. Frontend menampilkan animasi sukses hanya setelah backend mengonfirmasi commit.

Jangan mempercayai jumlah stamp yang dikirim frontend.

6.3 Reward redemption

Reward dapat berupa:

- Free item.
- Persentase diskon.
- Fixed discount.
- Voucher.
- Custom benefit.

Gunakan flow reservation dan confirmation:

1. Customer/staff meminta redemption.
2. Backend memvalidasi reward eligibility dan saldo.
3. Buat redemption reservation dengan token unik dan expiration.
4. Staff memvalidasi token.
5. Backend melakukan confirmation secara atomic.

6. Ledger mencatat redemption.
7. Token tidak dapat dipakai ulang.

Tangani:

- Reservation expired.
- Cancellation.
- Double scan.
- Concurrent redemption.
- Retry request.
- Reward stock/quantity jika ada.
- Subscription suspended.
- Reversal hanya melalui proses terkontrol dan audit.

7. Animations dan interaction

Teknologi:

- Lottie/DotLottie untuk animasi stamp, sparkle, reward unlock.
- CSS transitions/keyframes untuk hover, press, scale, bounce, fade, progress.
- Rive dapat dipertimbangkan kemudian untuk animasi stateful yang lebih kompleks.
- GSAP tidak wajib untuk MVP.

Animasi yang dibutuhkan:

1. Empty stamp slot.
2. Stamp success: pop/bounce, icon masuk, subtle sparkle.
3. Progress update.
4. Reward unlocked: celebratory animation.
5. Redemption success.
6. Loading/error states.

Penting:

- Animasi sukses hanya berjalan setelah server mengonfirmasi operasi berhasil.
- Jika request gagal, tampilkan error state, bukan success animation.
- Hormati `prefers-reduced-motion`.
- Jangan memuat animasi besar berulang; gunakan cache/static assets.
- Jangan menjadikan animasi sebagai satu-satunya indikator status.

8. Referral program

Referral harus didukung sebagai modul tersendiri. Prioritas MVP dapat ditentukan, tetapi database/domain harus siap.

8.1 Merchant referral

Customer lama mengundang customer baru untuk bergabung ke program merchant tertentu.

Contoh:

- Referrer A membagikan kode `ILHAMABC`.
- Customer B membuka `bunus.app/m/abc-coffee?ref=ILHAMABC`.
- B register/login dan join ABC Coffee.
- B melakukan qualifying first purchase.
- A mendapatkan 2 bonus stamps.
- B mendapatkan voucher diskon 10%.

Referral reward configurable per merchant:

- Stamp.
- Points.
- Discount voucher.
- Custom reward.

Merchant dapat mengatur:

- Enable/disable referral.
- Reward untuk referrer.
- Reward untuk referred customer.
- Qualifying event: registration+verified, first qualifying purchase, atau second purchase.
- Periode campaign.
- Batas reward dan eligibility.
- Branch/program scope.

Default yang direkomendasikan:

- Reward baru diberikan setelah customer baru melakukan transaksi pertama yang memenuhi syarat.
- Referral code unik per customer per merchant/program.
- Referral link otomatis.
- QR referral opsional.
- Referral attribution dikunci ketika customer join program.

8.2 Platform referral

Referral untuk mengajak pengguna baru ke Bunus secara keseluruhan dapat dikembangkan kemudian. Pisahkan dari merchant referral.

8.3 Fraud prevention

- Self-referral dilarang.
- Satu referral attribution per customer per program sesuai policy.
- Idempotency untuk reward.
- Deteksi akun/perangkat/IP mencurigakan secara proporsional dan sesuai privasi.
- Reward menunggu qualifying event.
- Jika transaksi dibatalkan, reward dapat direversal.
- Status referral: `PENDING`, `QUALIFIED`, `REWARDED`, `REVERSED`, `REJECTED`.

- Audit trail wajib.
- Jangan mengandalkan IP/device fingerprint sebagai satu-satunya bukti fraud.

9. Merchant Dashboard

Merchant mengakses dashboard melalui `bunus.app/merchant`.

9.1 Overview

Tampilkan KPI:

- Total members.
- Active members.
- Stamps issued.
- Points earned (jika program points aktif).
- Rewards unlocked/redeemed.
- Referral conversions.
- Member growth.
- Activity chart.
- Recent activity.
- Branch filter dan date range.

9.2 Menu

- Overview.
- Loyalty Programs.
- Members.
- Stamps & Points / Ledger.
- Rewards.
- Redemptions.
- Branches.
- Staff & Roles.
- Transactions.
- Campaigns (fase lanjutan).
- Referral Program.
- Analytics.
- Subscription & Billing.
- Notifications.
- Settings.

9.3 Merchant onboarding

MVP:

1. Super Admin membuat merchant.
2. Admin memasukkan data merchant dan memilih plan/trial.
3. Sistem mengirim invitation link ke Merchant Owner.
4. Owner membuat password sendiri.
5. Owner melengkapi profil, logo, cabang, loyalty program, reward, dan staff.
6. Merchant dapat mulai menggunakan sistem setelah status subscription/entitlement mengizinkan.

Jangan pernah membuat password lalu mengirim password plaintext melalui email.

Fase berikutnya:

- Self-registration merchant.
- Pilih plan.
- Pembayaran.
- Merchant workspace dibuat otomatis setelah payment terverifikasi.

9.4 Merchant branding

Merchant dapat mengatur:

- Nama bisnis.
- Logo.
- Warna tema terbatas/terkontrol.
- Deskripsi.
- Public slug.
- Branch information.
- Program name dan reward presentation.

Jangan mengizinkan arbitrary custom CSS merchant pada MVP.

10. Super Admin Dashboard

Akses: `bunus.app/admin`.

Fungsi:

- Platform overview.
- Merchant management.
- Create/invite merchant.
- Merchant status management.
- Subscription plans.
- Subscription management.
- Invoices.
- Payment transactions.
- Manual extension/adjustment.
- Trial management.

- Platform analytics.
- Notification logs.
- System settings.
- Audit logs.
- Support tools.
- Controlled impersonation/view-as-merchant (opsional).

KPI:

- Total merchants.
- Active subscriptions.
- Trial merchants.
- Expiring soon.
- Past due.
- Grace period.
- Suspended.
- MRR/ARR jika data billing sudah memadai.
- Payment success/failure.
- Platform usage.

Semua tindakan Super Admin yang mengubah merchant/subscription harus tercatat dalam audit log dengan actor, target, before/after, timestamp, dan reason bila relevan.

11. Subscription, billing, payment, dan automatic suspension

Subscription adalah modul platform-level dan harus dipisahkan dari Loyalty Engine.

11.1 Subscription plans

Plan dan harga harus configurable melalui Super Admin, bukan hardcoded.

Plan dapat menentukan:

- Harga monthly/annual.
- Jumlah branch.
- Jumlah staff.
- Jumlah program loyalty.
- Member limit (jika diterapkan).
- Feature entitlements.
- Notification quota.
- Support level.
- Trial eligibility.

Contoh fitur plan harus dapat diatur; jangan menganggap harga final telah ditentukan.

11.2 Billing model

Dua mode yang perlu didukung:

- Manual renewal: merchant membayar invoice setiap periode.
- Automatic recurring: pembayaran otomatis jika gateway dan metode pembayaran mendukung.

MVP dapat dimulai dengan manual invoice/payment, tetapi model data harus mendukung recurring.

11.3 Reminder schedule

Default rekomendasi:

- H-7: reminder pertama.
- H-3: reminder kedua.
- H-1: final reminder.
- H+0: jatuh tempo; status menjadi `PAST_DUE`.
- H+1 sampai H+7: `GRACE_PERIOD`.
- Setelah grace period berakhir: automatic suspension.

Semua schedule harus configurable dan idempotent agar scheduler tidak mengirim email berulang kali.

11.4 Subscription statuses

- `TRIAL`
- `ACTIVE`
- `EXPIRING_SOON` (dapat berupa derived status)
- `PAST_DUE`
- `GRACE_PERIOD`
- `SUSPENDED`
- `CANCELLED`
- `EXPIRED`

Definisikan status transitions secara eksplisit. Jangan mencampur status invoice dengan status subscription.

11.5 Saat suspended

Rekomendasi perilaku:

- Merchant Owner masih dapat login.
- Dapat melihat invoice dan status pembayaran.
- Dapat melakukan pembayaran.
- Dashboard menjadi read-only untuk data operasional.
- Tidak dapat issue stamp.
- Tidak dapat redeem reward.
- Tidak dapat menambah customer/membership.
- Tidak dapat menambah staff/branch atau mengubah program.
- Customer melihat pesan program sementara tidak tersedia.

- Data merchant tidak langsung dihapus.
- Retensi data dan deletion policy harus ditetapkan secara legal/operasional; 90 hari setelah suspension pernah dipertimbangkan sebagai contoh, bukan keputusan final.

11.6 Reactivation

Default rekomendasi: otomatis setelah payment webhook diverifikasi dan subscription diperbarui.

Payment webhook:

- Verifikasi signature.
- Validasi invoice ID, merchant, nominal, currency, status, dan payment reference.
- Idempotency untuk webhook duplicate/retry.
- Jangan memperpanjang subscription hanya karena browser redirect sukses.
- Catat payment event dan subscription event.
- Pastikan pembayaran yang sama tidak diproses dua kali.

Payment gateway kandidat untuk Indonesia: Xendit, Midtrans. Untuk pasar global, Stripe dapat dipertimbangkan berdasarkan eligibility negara bisnis. Harga/fitur harus diverifikasi saat integrasi.

12. Notification & Communication Engine

Pisahkan channel provider dari domain logic.

Channel:

- Email.
- WhatsApp.
- SMS.
- In-app notifications.

Use cases:

- Email verification.
- Password reset.
- Login/security notification.
- Stamp issued.
- Reward unlocked.
- Redemption confirmation.
- Stamp expiration reminder.
- Subscription reminders.
- Invoice/payment receipt.
- Referral qualified/rewarded.

12.1 Email

Gunakan sender domain sendiri: `noreply@bunus.app` atau alamat resmi yang dipilih.

SMTP hosting dapat digunakan untuk awal jika provider mengizinkan dan deliverability teruji. Buat adapter agar dapat berpindah ke Resend, Amazon SES, Brevo, atau provider lain.

Konfigurasi DNS email:

- SPF.
- DKIM.
- DMARC.

Monitor delivery, bounce, complaint, retry, suppression, dan provider error.

12.2 WhatsApp/SMS

Opsional; ada biaya provider/Meta/operator. Jangan mengirim tanpa consent yang sesuai.

Provider kandidat:

- OTP.ID.
- Mekari Qontak.
- Infobip.
- Twilio.

Biaya harus dianggap variabel dan diperiksa kembali sebelum keputusan vendor. Pisahkan biaya provider dan biaya Meta jika berlaku.

12.3 Notification consent

Pisahkan:

- Transactional notifications.
- Marketing/promotional notifications.

Persetujuan menerima notifikasi transaksi tidak otomatis berarti consent untuk promosi. Simpan consent, timestamp, source, channel, dan status unsubscribe.

12.4 Usage/billing model

Opsi monetisasi:

- Email transactional dasar termasuk subscription.
- WhatsApp/SMS memiliki kuota atau usage-based charge.
- Super Admin dapat melihat usage, delivery, failure, dan biaya per channel/merchant.
- Merchant tidak boleh melihat penggunaan merchant lain.

13. Brand and Design System

13.1 Brand

- Name: Bunus.
- Domain: bunus.app.

- Tagline: “Every visit brings a little more.”
- Personality: modern, friendly, premium, trustworthy.
- Mascot: geometric bunny/rabbit.
- Hindari kesan terlalu childish/game-like atau terlalu corporate banking.

Logo concept:

- Bunny + B.
- Rounded-square app icon.
- Emerald background.
- White/minimal bunny face.
- Yellow sparkle accent.
- Wordmark “Bunus”.
- Variants: primary horizontal, icon-only, monochrome, white/reversed.

Logo yang dihasilkan secara visual harus dibuat ulang sebagai vector/SVG production asset sebelum dipakai final; jangan menganggap ilustrasi raster sebagai master logo.

13.2 Color palette

Primary:

- Emerald 800: `#0B6B4F`
- Emerald 600: `#10B981`
- Emerald 100: `#D1FAE5`

Accent:

- Amber 400: `#FBBF24`
- Coral 400: `#FB7185`

Neutral:

- Gray 900: `#111827`
- Gray 700: `#374151`
- Gray 500: `#6B7280`
- Gray 300: `#D1D5DB`
- Gray 100: `#F3F4F6`
- White: `#FFFFFF`

Semantic:

- Success: `#16A34A`
- Warning: `#D97706`
- Error: `#DC2626`
- Info: `#2563EB`

Check WCAG 2.2 AA contrast. Do not use color as the only status indicator.

13.3 Typography

- Poppins: headings, brand, navigation, buttons.
- Inter: body, tables, numerical data, transaction records.
- Poppins weights: 400, 500, 600, 700.
- Inter weights: 400, 500, 600, 700.
- Use tabular numerals for balances, KPIs, timestamps, and amounts.

Type scale:

- Hero: 40–48 px desktop; 30–36 px mobile, 700.
- H1: 32 desktop / 28 mobile, 700.
- H2: 24 / 22, 700.
- H3: 20 / 18, 600.
- H4: 18 / 16, 600.
- Body large: 16, 400.
- Body: 14, 400.
- Small: 12, 400.
- Caption: 11, 500.
- KPI: 28–36 desktop; 28–32 mobile, 700.
- Line-height body 1.4–1.6; headings 1.2–1.3.

13.4 Component dimensions

- Button height: 44–48 px.
- Input height: 44–48 px.
- Card radius: 12–16 px.
- Button radius: 8–10 px.
- Input radius: 8 px.
- Modal radius: 16 px.
- Border: 1 px neutral.
- Desktop sidebar: 240–260 px.
- Mobile page padding: 16 px.
- Desktop page padding: 24–32 px.
- Standard icons: 18–24 px.
- Spacing scale based on 4 px.

13.5 Iconography

Use Lucide outline icons, generally 2 px stroke, rounded, consistent size. Do not mix unrelated icon styles.

Suggested:

- Home: `house`
- Merchant: `store`
- Rewards: `gift`
- Stamp: `stamp`

- Points: `coins`
- Voucher: `ticket`
- Members: `users`
- Referral: `users-round`
- Analytics: `chart-no-axes-combined`
- Settings: `settings`
- Subscription: `credit-card`
- Notifications: `bell`

13.6 Two visual modes

Customer PWA:

- More friendly and engaging.
- Bunny mascot can appear.
- Stamp/reward celebration.
- Emerald and mint.
- Rounded cards.

Merchant/Super Admin:

- Professional, clean, data-oriented.
- Minimal mascot usage.
- White/light neutral background.
- Emerald navigation/primary actions.
- Clear tables and analytics.

Merchant may customize logo and controlled theme colors in later implementation. Do not allow arbitrary CSS.

14. Hosting, domain, and deployment architecture

14.1 Domain

Primary domain selected: `bunus.app`.

Use one domain and path routes for MVP. Do not buy separate domains for customer/merchant/staff.

Cloudflare can be used for DNS, CDN, SSL/WAF features, subject to configuration. Domain account must be controlled by product owner, MFA enabled, auto-renew enabled, recovery methods secured.

14.2 Hosting requirement

Application is Node.js-based; PHP is prohibited.

Possible hosting:

- Managed Node.js hosting if it supports NestJS process, MySQL connectivity, environment variables, SSH/deployment, cron, logs, and adequate limits.

- Linux VPS for more control and portability.
- cPanel only if Node.js Application Manager/Passenger and all required capabilities are explicitly supported. Do not assume generic cPanel hosting supports Node.js.

14.3 Initial sizing target

For a small pilot:

- 2 vCPU.
- 4 GB RAM minimum; 8 GB preferred if backend and MySQL share the server.
- 80–100 GB NVMe/SSD.
- 1–4 TB monthly bandwidth is a generous initial target for a small pilot; actual usage must be measured.
- Ubuntu Server LTS.
- Nginx reverse proxy.
- NestJS Node.js LTS.
- MySQL 8.
- Docker/Docker Compose on VPS.
- Cron/background scheduler.
- External backups.
- Monitoring.

These are planning targets, not guaranteed capacity. Load-test and monitor before scaling.

14.4 Development and scaling

Development:

- Local environment.
- Local MySQL/container.
- GitHub.
- Staging environment.

Pilot:

- One VPS or managed Node host.
- App and DB can initially share host if resources are sufficient.
- Polling for dashboard updates; no Redis/WebSocket required initially.
- External object storage for logos/images.
- Daily external DB backup.

Growth:

- Separate DB if metrics require it.
- Redis for cache/queue when needed.
- Background workers.
- WebSocket/SSE adapter if real-time needs justify it.
- Object storage/CDN.
- Monitoring and alerting.

14.5 Portability requirements

- Modular monolith.
- Domain logic independent from hosting provider.
- Infrastructure adapters for email, storage, payment, queue, cache, notification, and event delivery.
- MySQL as portable relational DB.
- Prisma migrations committed to Git.
- UUID/public IDs; never expose sequential internal IDs as public identifiers.
- No persistent uploads on local app filesystem.
- Environment variables for configuration.
- No hardcoded absolute host paths or provider-specific URLs.
- Stateless application instances.
- Session strategy that supports multi-instance deployment.
- Dockerfile and docker-compose for local/staging/VPS.
- cPanel deployment must not require Docker.
- External backups and restore testing.
- Migration plan from managed/cPanel to VPS without business logic rewrite.

14.6 Backup and recovery

- Daily encrypted database backup to external storage.
- Retention policy (e.g. daily and weekly copies) to be finalized.
- Consider binary logs/PITR when production needs justify it.
- Backup application configuration and migration records.
- Test restore periodically.
- Define RPO/RTO before commercial scale.
- Never rely solely on backup stored on same VPS.

15. Suggested codebase structure

Use a monorepo or clearly separated applications. Suggested starting structure:

```
bunus/
  apps/
    api/                  # NestJS backend
    web/                  # Vue 3 customer/merchant/admin PWA
  packages/
    shared-types/
    validation/
    api-client/
    ui/
  prisma/
    schema.prisma
    migrations/
    seed/
  infrastructure/
    docker/
    nginx/
    scripts/
  tests/
    unit/
    integration/
    e2e/
```

```
  docs/
    architecture/
    api/
    database/
    runbooks/
  .env.example
  docker-compose.yml
  package.json
  README.md
```

Backend modules:

- Auth
- Users
- Customers
- Merchants/Tenants
- Branches
- Memberships
- LoyaltyPrograms
- Stamps
- Points
- Ledger
- Rewards
- Redemptions
- Referrals
- Subscriptions
- Plans
- Invoices
- Payments
- Notifications
- Audit
- Analytics
- Files/Storage

Frontend areas:

- Public website.
- Customer PWA.
- Merchant dashboard.
- Staff dashboard.
- Super Admin dashboard.

Keep module boundaries clear. Do not split into microservices prematurely.

16. Suggested database entities

This is a conceptual starting point; normalize and refine before implementation.

Identity and access

- `users`
- `user_credentials` or auth identity tables
- `customers`
- `merchant_users`
- `roles`
- `permissions`
- `role_permissions`
- `user_role_assignments`
- `sessions`
- `email_verification_tokens`
- `otp_challenges`
- `password_reset_tokens`

Merchant and branches

- `merchants`
- `merchant_profiles`
- `merchant_settings`
- `branches`
- `staff_branch_assignments`

Loyalty

- `loyalty_programs`
- `loyalty_program_versions`
- `program_memberships`
- `merchant_memberships` (if distinct)
- `earning_rules`
- `expiration_policies`
- `stamp_lots` or equivalent expiry-aware units
- `loyalty_ledger_entries`
- `balance_projections`
- `rewards`
- `reward_redemptions`
- `redemption_reservations`
- `qr_tokens`

Referral

- `referral_programs`
- `referral_codes`
- `referral_attributions`
- `referral_rewards`

- `referral_events`

Subscription and payments

- `subscription_plans`
- `plan_entitlements`
- `subscriptions`
- `subscription_invoices`
- `payment_transactions`
- `payment_webhook_events`
- `subscription_events`

Notifications and audit

- `notification_templates`
- `notification_preferences`
- `notification_consents`
- `notification_jobs`
- `notification_logs`
- `audit_logs`
- `outbox_events`

Important:

- Define tenant ownership for every merchant-scoped entity.
- Add composite unique constraints where required.
- Use foreign keys.
- Use decimal types for monetary amounts; never floating point.
- Store currency explicitly.
- Use UTC timestamps.
- Add indexes for tenant, membership, program, status, and date filters.
- Use soft-delete only where appropriate; ledger/payment/audit should be retained and corrected through compensating entries rather than destructive edits.
- Keep payment provider IDs and webhook idempotency keys unique.

17. API design conventions

Base: `/api/v1`.

- REST conventions.
- DTO validation server-side.
- Consistent error envelope.
- Pagination and filtering.
- Authenticated endpoints by default.
- Explicit permission guards.

- Idempotency keys for financial/loyalty mutations.
- Rate limiting for auth, OTP, QR, referral, and redemption.
- OpenAPI/Swagger documentation for internal and integration use.
- Never return cross-tenant records.

Example route groups (not final endpoint contract):

- `/auth/*`
- `/customer/me`
- `/customer/memberships`
- `/merchants/*`
- `/merchant/programs`
- `/merchant/members`
- `/staff/scan`
- `/staff/stamps/issue`
- `/rewards`
- `/redemptions`
- `/referrals`
- `/subscriptions`
- `/invoices`
- `/payments/webhooks/:provider`
- `/admin/merchants`
- `/admin/subscriptions`
- `/admin/plans`

18. Security, privacy, and reliability requirements

Mandatory:

- Tenant isolation at service/repository/query level.
- RBAC/permission guards.
- Secure password hashing.
- OTP hashing, expiration, attempt limits.
- CSRF protection where cookie auth requires it.
- XSS prevention and safe rendering.
- CORS configured narrowly.
- Secure cookies and HTTPS.
- Rate limiting.
- QR token expiration and replay prevention.
- Idempotency for stamp, redemption, referral reward, payment webhook.
- Atomic database transactions.
- Audit logs for sensitive operations.
- Secrets not committed.

- Database least-privilege accounts.
- Encryption in transit; protect backups at rest.
- Input validation and output encoding.
- PII minimization.
- Customer consent and notification preferences.
- Data export/deletion policy aligned with applicable law.
- Do not expose customer membership across merchants.
- Do not let staff access other branches without permission.

Reliability:

- Outbox pattern for events/notifications after DB commit.
- Retry with backoff for provider calls.
- Dead-letter/error visibility for failed jobs.
- Idempotent scheduled jobs.
- Health checks and structured logs.
- Monitoring of CPU, RAM, disk, DB connections, API latency, error rate, email delivery, payment webhook failures.

19. MVP scope and phased roadmap

Phase 0 — Foundation

- Project skeleton and CI.
- Auth and RBAC.
- Tenant model.
- Merchant and branch setup.
- Customer global account.
- Design system.
- Database migrations.
- Audit framework.

Phase 1 — Stamp MVP

- Merchant onboarding via Super Admin invitation.
- Merchant dashboard.
- Staff dashboard.
- Customer PWA.
- Global customer account.
- Join merchant via QR/link.
- Stamp program setup.
- Dynamic customer QR.
- Staff stamp issuance.
- Ledger and expiry policy.
- Reward catalog.

- Redemption reservation/confirmation.
- Customer history and progress.
- Basic analytics.
- Lottie/CSS stamp animation.
- Basic email verification and notifications.
- Basic subscription plans and manual invoice/payment tracking.
- Automatic subscription lifecycle/suspension.

Phase 2 — Growth features

- Points program.
- Merchant referral.
- WhatsApp/SMS providers.
- Advanced campaign tools.
- Automated payment/recurring billing.
- Advanced analytics.
- More notification preferences.
- Customer segmentation.
- Birthday/referral campaigns.

Phase 3 — Scale

- Multiple merchant programs.
- Custom merchant domains.
- White-label options.
- POS/API integrations.
- Redis/queue workers.
- WebSocket/SSE.
- Advanced fraud detection.
- Platform referral.
- Internationalization and multi-currency.
- Advanced billing/usage metering.

Do not build every Phase 2/3 feature before the Stamp MVP works reliably.

20. QA strategy and acceptance criteria

Testing is a first-class requirement. Prioritize API/integration tests for concurrency and tenant isolation.

20.1 Authentication

- Customer can register and verify email.
- Expired OTP/link is rejected.
- OTP cannot be reused.

- OTP brute-force is rate-limited.
- Password reset works safely.
- Customer can join a second merchant without creating another global account.

20.2 Tenant isolation

- Merchant A cannot query Merchant B data by changing IDs.
- Merchant A staff cannot access Merchant B branches.
- Customer can see own memberships across merchants but no other customer's data.
- Merchant sees only its own members and loyalty activity.
- Super Admin cross-tenant action is audited.

20.3 Stamp

- Valid staff can issue stamp.
- Unauthorized staff cannot issue stamp.
- Duplicate request with same idempotency key does not double issue.
- Concurrent requests cannot exceed earning eligibility.
- Failed DB transaction creates no success animation and no partial ledger.
- Expired stamp is excluded from progress.
- New expiration policy applies only to new stamps by default.
- Ledger reconciliation matches balance projection.

20.4 Reward redemption

- Ineligible customer cannot redeem.
- Reservation token expires.
- Token cannot be redeemed twice.
- Concurrent redemption cannot overdraw balance.
- Cancellation/expiry releases reservation correctly.
- Suspended merchant cannot process redemption.

20.5 Points (when enabled)

- Earning formula is server-side.
- Points expiration works by lot/policy.
- Redemption cannot create negative balance.
- Ledger and balance reconcile.

20.6 Referral

- Referral code maps to correct merchant/program.
- Self-referral is rejected.
- Attribution is stable across login/register redirect.
- Reward only after qualifying event.
- Duplicate qualifying event does not double reward.
- Reversal works when qualifying transaction is reversed.

- Merchant cannot inspect other merchant referral records.

20.7 Subscription

- H-7/H-3/H-1 reminders sent once.
- Past due/grace/suspended transitions work.
- Suspended merchant can view invoice and pay.
- Suspended merchant cannot issue stamp or redeem reward.
- Valid payment webhook reactivates subscription once.
- Duplicate webhook does not extend subscription twice.
- Data remains intact after suspension.

20.8 Performance and operations

- Load test login, QR validation, stamp issue, and dashboard.
- Measure API p95 latency, error rate, DB connection use, CPU/RAM.
- Verify backup restore.
- Verify deployment and migration rollback strategy.
- Test on mobile viewport and PWA installation.
- Test reduced-motion mode and accessibility.

21. Decisions confirmed vs. decisions still open

Confirmed product direction

- Product name: Bunus.
- Domain selected: `bunus.app`.
- PHP is explicitly prohibited.
- SaaS multi-merchant model.
- Customer global account.
- Customer can join multiple merchants with separate membership/benefits.
- Separate Customer PWA, Merchant Dashboard, Staff Dashboard, Super Admin Dashboard.
- NestJS + TypeScript, Vue 3 + TypeScript, MySQL 8, Prisma is the working implementation baseline.
- Modular monolith.
- Stamp program as first launch focus.
- Platform foundation should support Points.
- Merchant-configurable stamp expiration.
- Lottie/CSS stamp animation.
- Merchant onboarding initially via Super Admin invitation.
- Subscription reminder and automatic suspension.
- Email verification using Bunus-owned sender domain; SMTP/API provider abstraction.
- WhatsApp/SMS optional due to cost.
- Merchant referral concept.

- Emerald/mint brand palette.
- Poppins + Inter.
- Lucide outline icons.

Decisions requiring product owner confirmation

- Whether Points must launch in MVP or only be architecturally prepared.
- Exact earning rules supported in first release.
- Whether merchant can use both per-item duration and fixed-date expiration simultaneously.
- Final expiration behavior for legacy stamps when merchant changes policy.
- Exact trial duration (14 days was proposed).
- Grace period (7 days was proposed).
- Whether subscription starts manual billing only or includes recurring from MVP.
- Final subscription plan names/prices/limits.
- Final payment gateway and merchant country/legal entity requirements.
- Whether suspended customer can view historic membership read-only.
- Data retention/deletion after suspension.
- Final OTP method (email link, email OTP, or both).
- Email delivery provider.
- WhatsApp/SMS provider and quota model.
- Referral qualification event and rewards.
- Final logo/vector assets and whether Bunny mascot should be used in all customer surfaces.
- Hosting vendor and exact plan.

22. First implementation tasks for coding agent

Start in this order:

1. Create monorepo and tooling.
2. Create NestJS API and Vue 3 app.
3. Configure MySQL 8 + Prisma.
4. Add Docker Compose for local development.
5. Add lint, format, typecheck, unit test, integration test, and CI.
6. Implement auth identities, customer global account, merchant, merchant users, RBAC.
7. Implement tenant context and isolation tests before business features.
8. Implement branches and merchant onboarding.
9. Implement loyalty program and stamp rules.
10. Implement immutable ledger and expiry-aware stamp issuance.
11. Implement reward catalog and redemption reservation/confirmation.
12. Implement Customer PWA membership list and merchant dashboard.
13. Implement staff QR scan and issuance.
14. Add Lottie/CSS animations after API correctness.

15. Implement subscription plans, invoices, reminders, grace period, suspension, and payment adapter.
16. Implement referral module.
17. Add email/notification provider adapters.
18. Add analytics, monitoring, backup, and deployment runbooks.

Do not start by building only static dashboards. First establish the data model, authorization, ledger correctness, and transaction behavior. UI should consume real API contracts.

Final instruction

Build Bunus.app as a production-minded, secure, portable, multi-tenant SaaS. Keep the first release focused on a reliable Stamp loyalty experience while making the architecture extensible for Points, Referral, notifications, and subscription growth.

PHP is not allowed anywhere in the application runtime or backend implementation.